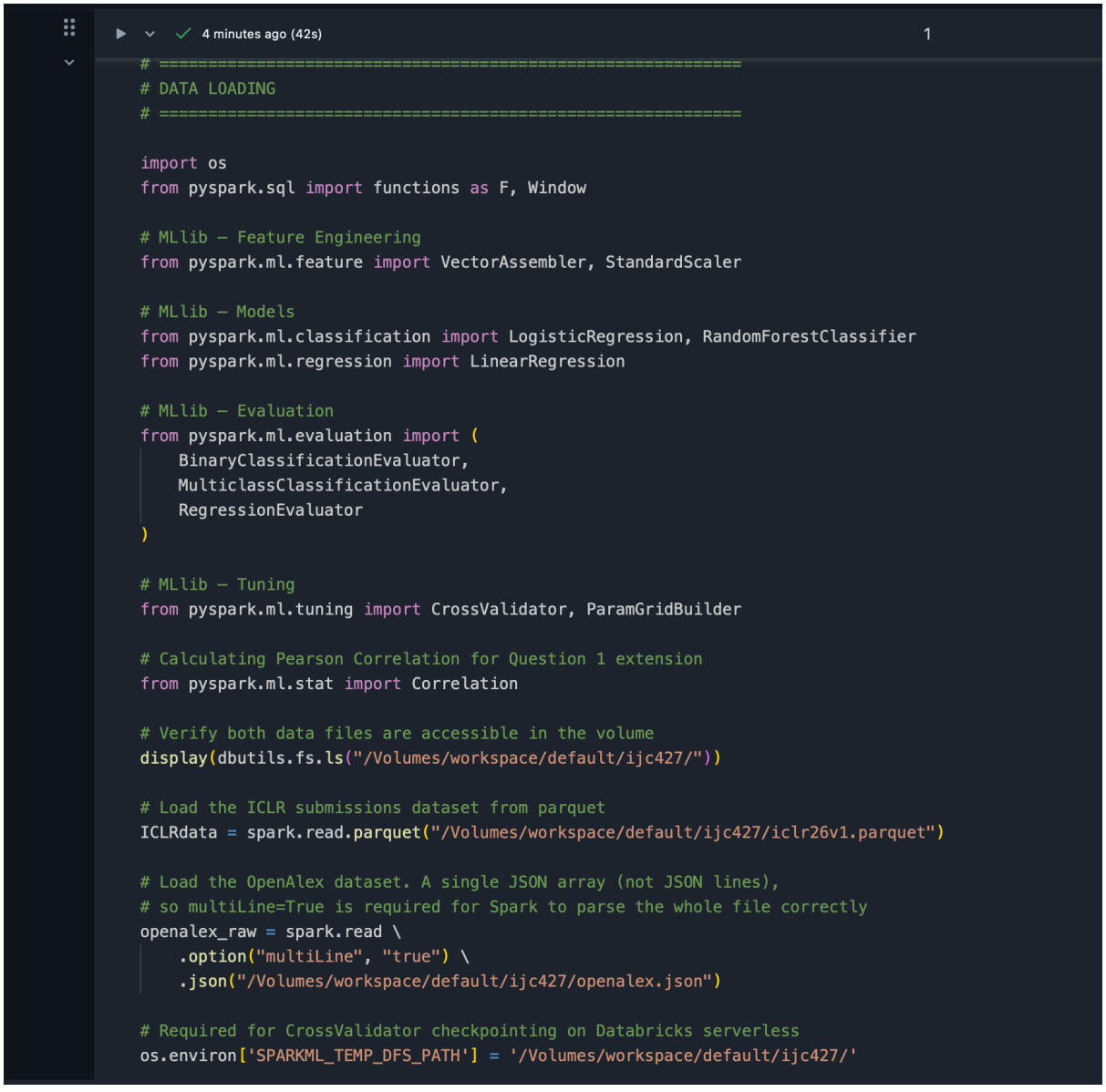
Big Data Analysis of ICLR Submissions for the ICLR Programme Committee

Cailey Coghlan
MSc Data Science, University of Sheffield
Module: IJC427 Big Data Analytics
Word Count: 4,246

1. Setup:

The code was written in a Databricks serverless notebook and utilises PySpark DataFrames throughout. MLlib was employed in the extension analyses for questions 1 and 3. No manual configuration is required. By simply clicking ‘Run All’ in the notebook, all cells successfully run consecutively. Both datasets are loaded as DataFrames in cell 1. This ensures that only a single change is required if the code needs to be adjusted for later dataset releases. Finally, the code for the core questions were validated against two earlier dataset releases (iclr24v2 and iclr25v2) confirming that the code produces the expected reference outputs.



```
# Required for CrossValidator checkpointing on Databricks serverless
os.environ['SPARKML_TEMP_DFS_PATH'] = '/Volumes/workspace/default/ijc427/'
```

ICLRdata: pyspark.sql.connect.dataframe.DataFrame = [year: long, id: string ... 8 more fields]
openalex_raw: pyspark.sql.connect.dataframe.DataFrame = [apc_list: struct, apc_paid: struct ... 48 more fields]

Table +

	path	name	size	modificationTime
1	dbfs:/Volumes/workspace/default/ijc427/iclr24v2.parquet	iclr24v2.parquet	19656199	1774370026000
2	dbfs:/Volumes/workspace/default/ijc427/iclr25v2.parquet	iclr25v2.parquet	31710045	1774370032000
3	dbfs:/Volumes/workspace/default/ijc427/iclr26v1.parquet	iclr26v1.parquet	48993373	1774370037000
4	dbfs:/Volumes/workspace/default/ijc427/openalex.json	openalex.json	31234421	1774370253000

4 rows | 42.21s runtime Refreshed 3 minutes ago

2. Data Cleaning and Preparation

ICLR Submissions Dataset:

The iclr26v1 dataset is a scrape of submissions to the International Conference on Learning Representations (ICLR), covering 55,906 papers submitted across ten conference cycles from 2017 to 2026. Schema inspection confirmed that the schemas for all of the variables were correct. Notably, scores and keywords were loaded as array types, enabling higher order Spark functions to be applied directly without type casting. No duplicate submission IDs were found. Of the 55,906 rows, 20,289 had empty score arrays. Of the rows with empty score arrays, most of them had empty decision columns (19,673). This was consistent with the 19,673 unreviewed 2026 submissions present in the dataset. A cleaned DataFrame, ICLRdata_clean, contains both non-empty decision strings and non-empty score arrays, retaining 35,617 rows. A further 2,131 rows with empty keyword arrays were excluded in question 4.

OpenAlex Dataset:

The OpenAlex scrape contains 2,245 academic papers with publishing years spanning from 1988 to 2025. This is a subset of the OpenAlex database extracted to match ICLR submissions. The OpenAlex scrape is joined with iclr26v1 in questions 5 and 6 in order to retrieve citation counts and authorship information. After normalising titles to lowercase and removing excess whitespace, 123 titles appeared more than once in the OpenAlex scrape. Duplicate handling is addressed within each question's implementation section. 1,023 papers in the scrape had a null citation percentile value and were excluded in Question 6.

10:13 PM (12s) 2

```
# ============================================================
# DATA CLEANING
# ============================================================

# ── ICLR DATASET ──────────────────────────────────────────────

# Preview schema and row count to confirm correct loading
print("ICLR dataset preview:")
ICLRdata.show()
iclr_total_rows = ICLRdata.count()
print("Row count:", iclr_total_rows)
print("\nICLR dataset schema:")
ICLRdata.printSchema()

# Count null values across all columns simultaneously
print("\nNull value counts per column:")
ICLRdata.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c)
    for c in ICLRdata.columns
]).show()

# No duplicate IDs were found so no duplicate papers
duplicate_ids = ICLRdata.groupBy("id").count().filter(F.col("count") > 1).count()
print(f"Duplicate IDs: {duplicate_ids}")

# Confirm year range is complete with no unexpected values
iclr_null_years = ICLRdata.filter(F.col("year").isNull()).count()
print("Null years:", iclr_null_years)
print("\nDistinct years in dataset:")
ICLRdata.select("year").distinct().orderBy("year").show()
print("\nSubmission counts by year:")
ICLRdata.groupBy("year").count().orderBy("count", ascending=False).show(truncate=False)

# Audit decision strings to expose casing inconsistencies and empty string rows
iclr_empty_decisions = ICLRdata.filter(F.col("decision") == "").count()
print("Empty decision strings:", iclr_empty_decisions)
print("\nDecision string counts (all values):")
ICLRdata.groupBy("decision").count().orderBy("count", ascending=False).show(truncate=False)
```

```
# Check whether empty decision strings are concentrated in 2026 submissions
print("\nDecision breakdown for 2026 submissions:")
ICLRdata.filter(F.col("year") == 2026).groupBy("decision").count().orderBy("count", ascending=False).
show(truncate=False)

print("\nYear breakdown for papers with empty decision strings:")
ICLRdata.filter(F.col("decision") == "").groupBy("year").count().orderBy("year").show(truncate=False)

# Check scores array for nulls, empty arrays, zero values and intra array nulls
iclr_null_scores  = ICLRdata.filter(F.col("scores").isNull()).count()
iclr_empty_scores = ICLRdata.filter(F.size(F.col("scores")) == 0).count()
iclr_zero_scores  = ICLRdata.filter(F.array_contains(F.col("scores"), 0)).count()
iclr_null_in_arr  = ICLRdata.filter(F.exists(F.col("scores"), lambda x: x.isNull())).count()
print("Null score arrays:",       iclr_null_scores)
print("Empty score arrays:",      iclr_empty_scores)
print("Zero scores:",       iclr_zero_scores)
print("Null within array:", iclr_null_in_arr)

# Check keywords array for nulls and empty arrays. Used in Q4 to find spiking topics
iclr_null_keywords  = ICLRdata.filter(F.col("keywords").isNull()).count()
iclr_empty_keywords = ICLRdata.filter(F.size(F.col("keywords")) == 0).count()
print(f"Null keywords arrays: {iclr_null_keywords}")
print(f"Empty keywords arrays: {iclr_empty_keywords}")

# Confirm empty score arrays are systematic (withdrawn/desk-rejected), not random gaps
print("\nDecision breakdown for papers with empty score arrays:")
ICLRdata.filter(
    F.size(F.col("scores")) == 0
).groupBy("decision").count().orderBy("count", ascending=False).show(truncate=False)

# Remove papers with no recorded decision or no reviewer scores
ICLRdata_clean = ICLRdata.filter(
    (F.col("decision") != "") &
    (F.size(F.col("scores")) > 0)
)
iclr_clean_rows = ICLRdata_clean.count()
print("Cleaned row count:", iclr_clean_rows)
```

```
▶ ⌄ ✓ 10:13 PM (12s)                                                    2

# ── ICLR SUMMARY ──────────────────────────────────────────────────────

print("=" * 55)
print("ICLR DATA QUALITY SUMMARY")
print("=" * 55)
print(f"Duplicate IDs:            {duplicate_ids}")
print(f"Total rows loaded:        {iclr_total_rows}")
print(f"Null years:               {iclr_null_years}")
print(f"Empty decision strings:   {iclr_empty_decisions}")
print(f"Null scores arrays:       {iclr_null_scores}")
print(f"Empty scores arrays:      {iclr_empty_scores}")
print(f"Zero scores:              {iclr_zero_scores}")
print(f"Null elements in scores arrays:  {iclr_null_in_arr}")
print(f"Null keywords arrays:     {iclr_null_keywords}")
print(f"Empty keywords arrays:    {iclr_empty_keywords}")
print(f"Rows retained after clean:{iclr_clean_rows}")
print(f"Rows removed:             {iclr_total_rows - iclr_clean_rows}")
print("=" * 55)

# ── OPENALEX DATASET ──────────────────────────────────────────────────
# openalex.json is a single JSON array, not JSON lines
# multiLine=True is required so Spark reads the whole file as one document
# rather than attempting to parse each line independently.

print("\nOpenAlex dataset preview:")
openalex_raw.show()

# Inspecting the schema for the variables of interest.
print("\nOpenAlex schema for key fields:")
openalex_raw.select("title", "contains_iclr", "authorships", "cited_by_count").printSchema()

# Identifying the span of the publication years in the dataset
print("\nOpenAlex publication year range:")
openalex_raw.agg(
    F.min("publication_year").alias("earliest"),
    F.max("publication_year").alias("latest")
).show()
```

```
# Null and empty string check on title: used as the join key throughout Q5 and Q6
print("\nNull counts for title and authorships:")
openalex_raw.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c)
    for c in ["title", "authorships"]
]).show()
empty_titles = openalex_raw.filter(F.trim(F.col("title")) == "").count()
print(f"Empty or whitespace-only titles: {empty_titles}")

# # Check for duplicate titles after normalising to lowercase and removing excess whitespace.
oa_title_counts = openalex_raw \
    .select(F.trim(F.lower(F.col("title"))).alias("norm_title")) \
    .groupBy("norm_title") \
    .count()
oa_duplicate_titles = oa_title_counts.filter(F.col("count") > 1).count()
print(f"Duplicate normalised titles in OpenAlex: {oa_duplicate_titles}")

# Check authorships array for null and empty entries at the top level
null_authorships  = openalex_raw.filter(F.col("authorships").isNull()).count()
empty_authorships = openalex_raw.filter(F.size(F.col("authorships")) == 0).count()
print(f"Null authorships arrays: {null_authorships}")
print(f"Empty authorships arrays: {empty_authorships}")

# Check coverage of the nested countries array: used in Q5 extension for
# geographic analysis of submission volume, acceptance rate, and reviewer scores
authorships_exploded = openalex_raw.select(F.explode("authorships").alias("authorship"))
empty_countries = authorships_exploded.filter(
    F.size(F.col("authorship.countries")) == 0
).count()
print("Authorships with empty countries array:", empty_countries)
```

```
# Check citation_normalized_percentile.value for nulls and zeros: Q6 uses this
# field to identify papers in the 95th citation percentile
oa_null_percentile = openalex_raw.filter(
    F.col("citation_normalized_percentile.value").isNull()
).count()
oa_zero_percentile = openalex_raw.filter(
    F.col("citation_normalized_percentile.value") == 0
).count()
print(f"Null citation_normalized_percentile.value: {oa_null_percentile}")
print(f"Zero citation_normalized_percentile.value: {oa_zero_percentile}")

# Check publication_year is populated: confirms the field is usable for year based analysis
print("\nDistinct publication years in OpenAlex:")
openalex_raw.select("publication_year").distinct().orderBy("publication_year").show()

# ── OPENALEX SUMMARY ──────────────────────────────────────────────────

print("=" * 55)
print("OPENALEX DATA QUALITY SUMMARY")
print("=" * 55)
total_records = openalex_raw.count()
print(f"Total records loaded:        {total_records}")
print(f"Duplicate normalised titles: {oa_duplicate_titles}")
print(f"Empty/null titles:           {empty_titles}")
print(f"Null authorships arrays:     {null_authorships}")
print(f"Empty authorships arrays:    {empty_authorships}")
print(f"Authorships empty countries: {empty_countries}")
print("=" * 55)
print(f"Null percentile values:      {oa_null_percentile}")
print(f"Zero percentile values:      {oa_zero_percentile}")
```

ICLR dataset preview:

year	id	title	abstract	authors	author_ids	decision	scores	keywords	labels
2017	B1-Hhnslg	Prototypical Netw...	A recent approach...	Jake Snell, Kevin...		Reject	[6, 4, 5]	[deep learning, t...	transfer learning
2017	B1-q5Pqxl	Machine Comprehen...	Machine comprehen...	Shuohang Wang, Ji...		Accept (Poster)	[6, 6, 7]	[natural language...	language models
2017	B16Jem9xe	Learning in Impli...	Generative advers...	Shakir Mohamed, B...		Invite to Worksho...	[8, 7, 6]	[unsupervised lea...	unlabeled
2017	B16dGcqlx	Third Person Imit...	Reinforcement lea...	Bradly C Stadie, ...		Accept (Poster)	[6, 5, 6]	[]	unlabeled
2017	B184E5qee	Improving Neural ...	We propose an ext...	Edouard Grave, Ar...		Accept (Poster)	[7, 9, 5]	[natural language...	language models
2017	B186cP9gx	Eigenvalues of th...	We look at the ei...	Levent Sagun, Leo...		Reject	[3, 4, 4, 4]	[optimization, de...	optimization
2017	B1E7Pwqgl	Cooperative Train...	This paper studie...	Jianwen Xie, Yang...		Reject	[6, 4, 3]	[unsupervised lea...	unlabeled
2017	B1ElR4cgg	Adversarially Lea...	We introduce the ...	Vincent Dumoulin,...		Accept (Poster)	[7, 8, 7]	[computer vision,...	semi-supervised l...
2017	B1G9tvcgx	Neural Machine Tr...	Although attentio...	Joji Toyama, Masa...		Reject	[4, 3, 3]	[]	unlabeled
2017	B1GOWV5eg	Learning to Repea...	Reinforcement Lea...	Sahil Sharma, Ara...		Accept (Poster)	[7, 8, 8]	[deep learning, r...	RL
2017	B1Igu2ogg	Efficient Vector ...	We present an eff...	Minmin Chen		Accept (Poster)	[7, 6, 7]	[natural language...	semi-supervised l...
2017	B1IzH7cxl	A Neural Stochast...	In this paper, we...	Rui Luo, Xiaojun ...		Reject	[6, 5, 5]	[deep learning, s...	unlabeled
2017	B1KBHtcel	Here's My Point: ...	One of the major ...	Peter Potash, Ale...		Reject	[4, 5, 5]	[natural language...	language models
2017	B1M8JF9xx	On the Quantitati...	The past several ...	Yuhuai Wu, Yuri B...		Accept (Poster)	[7, 7, 6]	[deep learning, u...	unlabeled
2017	B1MRcPclx	Query-Reduction N...	In this paper, we...	Minjoon Seo, Sewo...		Accept (Poster)	[7, 7, 7]	[natural language...	language models
2017	B1PA8fqeg	Multiagent System...	We propose a mult...	Hiroki Kurotaki, ...		Reject	[2, 1, 3]	[]	unlabeled
2017	B1TTpYKgx	On the Expressive...	We study the expr...	Maithra Raghu, Be...		Reject	[6, 3, 5]	[theory, deep lea...	unlabeled

only showing top 20 rows
Row count: 55906

ICLR dataset schema:

```
root
 |-- year: long (nullable = true)
 |-- id: string (nullable = true)
 |-- title: string (nullable = true)
 |-- abstract: string (nullable = true)
 |-- authors: string (nullable = true)
 |-- author_ids: string (nullable = true)
 |-- decision: string (nullable = true)
 |-- scores: array (nullable = true)
 |    |-- element: long (containsNull = true)
 |-- keywords: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- labels: string (nullable = true)
```

Null value counts per column:

year	id	title	abstract	authors	author_ids	decision	scores	keywords	labels
0	0	0	0	0	0	0	0	0	0

Duplicate IDs: 0
Null years: 0

```
Distinct years in dataset:
+----+
|year|
+----+
|2017|
|2018|
|2019|
|2020|
|2021|
|2022|
|2023|
|2024|
|2025|
|2026|
+----+
```

```
Submission counts by year:
+----+-----+
|year|count|
+----+-----+
|2026|19793|
|2025|11663|
|2024|7401 |
|2023|4955 |
|2022|3422 |
|2021|3009 |
|2020|2593 |
|2019|1569 |
|2018|1012 |
|2017|489  |
+----+-----+
```

```
Empty decision strings: 19673

Decision string counts (all values):
+-----------------------+-----+
|decision               |count|
+-----------------------+-----+
|                       |19673|
|Reject                 |17096|
|Withdrawn              |7594 |
|Accept (Poster)        |6173 |
|Accept (poster)        |1809 |
|Accept: poster         |1202 |
|Accept (Spotlight)     |775  |
|Accept (Oral)          |383  |
|Accept (spotlight)     |366  |
|Accept: notable-top-25%|280  |
|Desk rejected          |194  |
|Invite to Workshop Track|136  |
|Accept: notable-top-5% |91   |
|Accept (oral)          |86   |
|Accept (Talk)          |48   |
```

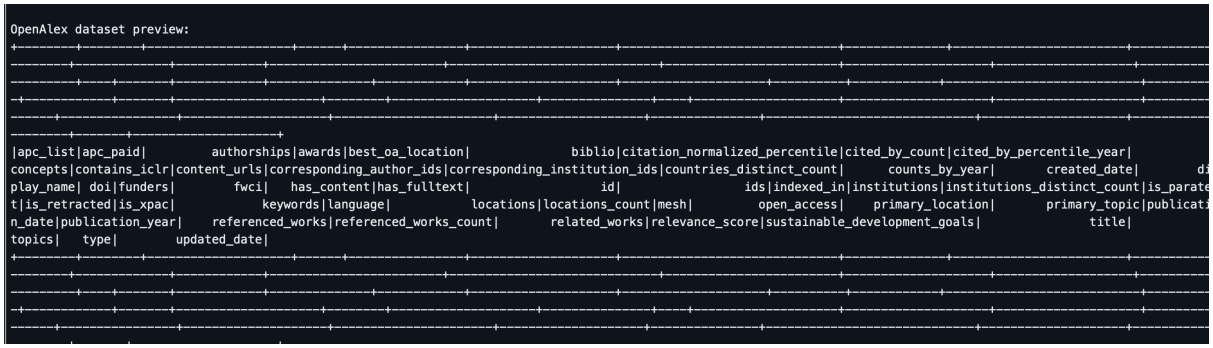
```
Decision breakdown for 2026 submissions:
+---------+-----+
|decision |count|
+---------+-----+
|         |19673|
|Withdrawn|120  |
+---------+-----+


Year breakdown for papers with empty decision strings:
+----+-----+
|year|count|
+----+-----+
|2026|19673|
+----+-----+
```

```
Null score arrays: 0
Empty score arrays: 20289
Zero scores: 0
Null within array: 0
Null keywords arrays: 0
Empty keywords arrays: 2131

Decision breakdown for papers with empty score arrays:
+-------------+-----+
|decision     |count|
+-------------+-----+
|             |19673|
|Withdrawn    |453  |
|Desk rejected|162  |
|Reject       |1    |
+-------------+-----+
```

```
Cleaned row count: 35617
============================================================
ICLR DATA QUALITY SUMMARY
============================================================
Duplicate IDs:             0
Total rows loaded:         55906
Null years:                0
Empty decision strings:    19673
Null scores arrays:        0
Empty scores arrays:       20289
Zero scores:               0
Null elements in scores arrays:  0
Null keywords arrays:      0
Empty keywords arrays:     2131
Rows retained after clean:35617
Rows removed:              20289
============================================================
```



```
OpenAlex schema for key fields:
root
 |-- title: string (nullable = true)
 |-- contains_iclr: boolean (nullable = true)
 |-- authorships: array (nullable = true)
 |    |-- element: struct (containsNull = true)
 |    |    |-- affiliations: array (nullable = true)
 |    |    |    |-- element: struct (containsNull = true)
 |    |    |    |    |-- institution_ids: array (nullable = true)
 |    |    |    |    |    |-- element: string (containsNull = true)
 |    |    |    |    |-- raw_affiliation_string: string (nullable = true)
 |    |    |-- author: struct (nullable = true)
 |    |    |    |-- display_name: string (nullable = true)
 |    |    |    |-- id: string (nullable = true)
 |    |    |    |-- orcid: string (nullable = true)
 |    |    |-- author_position: string (nullable = true)
 |    |    |-- countries: array (nullable = true)
 |    |    |    |-- element: string (containsNull = true)
 |    |    |-- institutions: array (nullable = true)
 |    |    |    |-- element: struct (containsNull = true)
```

```
 |    |    |    |    |-- element: string (containsNull = true)
 |    |    |-- institutions: array (nullable = true)
 |    |    |    |-- element: struct (containsNull = true)
 |    |    |    |    |-- country_code: string (nullable = true)
 |    |    |    |    |-- display_name: string (nullable = true)
 |    |    |    |    |-- id: string (nullable = true)
 |    |    |    |    |-- lineage: array (nullable = true)
 |    |    |    |    |    |-- element: string (containsNull = true)
 |    |    |    |    |-- ror: string (nullable = true)
 |    |    |    |    |-- type: string (nullable = true)
 |    |    |-- is_corresponding: boolean (nullable = true)
 |    |    |-- raw_affiliation_strings: array (nullable = true)
 |    |    |    |-- element: string (containsNull = true)
 |    |    |-- raw_author_name: string (nullable = true)
 |-- cited_by_count: long (nullable = true)
```

```
OpenAlex publication year range:
+--------+------+
|earliest|latest|
+--------+------+
|    1988|  2025|
+--------+------+


Null counts for title and authorships:
+-----+-----------+
|title|authorships|
+-----+-----------+
|    0|          0|
+-----+-----------+
```

```
Empty or whitespace-only titles: 0
Duplicate normalised titles in OpenAlex: 123
Null authorships arrays: 0
Empty authorships arrays: 4
Authorships with empty countries array: 2176
Null citation_normalized_percentile.value: 1023
Zero citation_normalized_percentile.value: 0
```

```
Distinct publication years in OpenAlex:
+----------------+
|publication_year|
+----------------+
|            1988|
|            1989|
|            1990|
|            1991|
|            1992|
|            1995|
|            1996|
|            1998|
|            1999|
|            2002|
|            2003|
|            2004|
|            2010|
|            2011|
|            2014|
|            2015|
|            2016|
```

```
|            2011|
|            2014|
|            2015|
|            2016|
|            2017|
|            2018|
|            2019|
+----------------+
only showing top 20 rows
======================================================
OPENALEX DATA QUALITY SUMMARY
======================================================
Total records loaded:        2245
Duplicate normalised titles: 123
Empty/null titles:           0
Null authorships arrays:     0
Empty authorships arrays:    4
Authorships empty countries: 2176
======================================================
Null percentile values:      1023
Zero percentile values:      0
```

Question 1: Paper Decision Counts and Predictive Modeling of Acceptance

Assumptions:

Empty decision strings indicate unreviewed submissions: Papers with empty decisions are assumed to be papers that have not been reviewed yet. This assumption holds, since the 19,673 papers with empty decisions are the unreviewed 2026 submissions identified in section 2 of this report. If any rows with empty decisions were in fact decided papers with missing records, the reported counts would underestimate the true totals.

Withdrawal is treated as a distinct unsuccessful outcome: It is assumed that most withdrawals occur due to reviewer feedback and the authors anticipating rejection. This makes withdrawals different from both accepted and outright rejected papers. This assumption holds for the majority of cases.

Implementation:

Papers were classified using a chained F.when().otherwise() expression in a single pass, with case-insensitive pattern matching handling casing inconsistencies such as 'Accept(Talk)' and 'Accept(talk)'. Counts were aggregated with groupBy('category').count().

```
10:32 PM (1s)                                                    3

# ============================================================
# QUESTION 1 — ACCEPTED, REJECTED AND WITHDRAWN PAPER COUNTS
# ============================================================
# Uses ICLRdata (not ICLRdata_clean) — decisions are needed but scores are not.


ICLR_q1 = ICLRdata.filter(F.col("decision") != "") \
    .withColumn("category",
        F.when(F.col("decision").rlike(
            "(?i)accept|oral|poster|spotlight|workshop|talk|notable"), "Accepted")
         .when(F.col("decision").rlike(
            "(?i)withdraw"), "Withdrawn")
         .otherwise("Rejected")
    )

# Count papers in each category
ICLR_q1.groupBy("category") \
     .count() \
     .orderBy("count", ascending=False) \
     .show()
```

Results:

```
ICLR_q1: pyspark.sql.connect.dataframe.DataFrame
+---------+-----+
| category|count|
+---------+-----+
| Rejected|17290|
| Accepted|11349|
|Withdrawn| 7594|
+---------+-----+
```

Discussion:

This question covered how many papers in the ICLR dataset were accepted, rejected, or withdrawn. Of the 36,233 papers with a recorded decision, 11,349 were accepted,

representing an acceptance rate of 31.3%. The most significant finding was that a notable proportion of papers with a known decision were withdrawn (7,594 or 21%). With 1 in 5 decided submissions withdrawn after receiving reviewer feedback, this burden will only grow as submission volume increases. Therefore, the ICLR committee should consider expanding the desk rejection criteria to remove weak submissions before they are allocated for full review. Alternatively, they could introduce a screening system that flags submissions likely to be withdrawn at an early stage, before reviewers commit their effort to a full review.

The core results established the distribution of outcomes but did not explain what drives acceptance decisions. To investigate this, a random forest classification model was built to determine whether reviewer scores alone can predict acceptance decisions, and to identify which specific aspects of scoring matter most.

Extension Assumption:

Withdrawn papers are treated as not accepted for classification purposes: The random forest model groups withdrawn and rejected papers into the same class. If a significant number of withdrawn papers were strong submissions pulled for reasons unrelated to their scores, the model would learn from incorrectly labelled data. This assumption holds for the majority of cases, since most withdrawals in the dataset follow reviewer feedback rather than decisions unrelated to paper quality.

Extension Implementation:

ICLRdata_clean was used as both scores and decisions are required. Five score-based features were assembled using VectorAssembler. A Pearson correlation matrix confirmed high collinearity between mean and minimum score ($r = 0.850$) and mean and maximum score ($r = 0.839$), motivating random forest over logistic regression which assumes feature independence. An 80/20 stratified split preserved the class imbalance. CrossValidator tuned maxDepth and numTrees via 3-fold cross-validation with AUC as the primary metric.

```
# ============================================================
# QUESTION 1 EXTENSION — PREDICTING ACCEPTANCE USING
# REVIEWER SCORES WITH RANDOM FOREST
# ============================================================
# Binary classification using Random Forest.
# Features are derived from the scores array; labels are 1=Accepted, 0=Not Accepted.
# Stratified 80/20 split preserves class proportions across train and test.
# CrossValidator tunes tree depth and number of trees via 3-fold cross-validation.

# Derive five score based features from the scores array
ICLRdata_features = ICLRdata_clean \
    .withColumn("mean_score",
        (F.aggregate(F.col("scores"), F.lit(0.0), lambda acc, x: acc + x.cast("double")) /
         F.size(F.col("scores")))
    ) \
    .withColumn("min_score",   F.array_min(F.col("scores")).cast("double")) \
    .withColumn("max_score",   F.array_max(F.col("scores")).cast("double")) \
    .withColumn("num_reviewers", F.size(F.col("scores"))) \
    .withColumn("std_score",
        F.sqrt(
            F.aggregate(
                F.col("scores"), F.lit(0.0),
                lambda acc, x: acc + (x.cast("double") - F.col("mean_score")) ** 2
            ) / F.size(F.col("scores"))
        )
    )

# Show first 5 rows with full descriptive column names
ICLRdata_features.select(
    F.col("mean_score").alias("Mean Score"),
    F.col("min_score").alias("Minimum Score"),
    F.col("max_score").alias("Maximum Score"),
    F.col("num_reviewers").alias("Number of Reviewers"),
    F.col("std_score").alias("Standard Deviation of Scores (Reviewer Disagreement)")
).show(5)
```

```
# Add binary label: 1.0 = Accepted, 0.0 = Not Accepted
# Withdrawn papers are treated as Not Accepted. They represent unsuccessful
# submissions regardless of the reason for withdrawal
ICLRdata_labelled = ICLRdata_features.withColumn("label",
    F.when(F.col("decision").rlike(
        "(?i)accept|oral|poster|spotlight|workshop|talk|notable"), 1.0)
     .otherwise(0.0)
)

feature_names  = ["mean_score", "min_score", "max_score", "num_reviewers", "std_score"]
feature_labels = ["Mean Score", "Minimum Score", "Maximum Score", "Number of Reviewers", "Standard
Deviation of Scores (Reviewer Disagreement)"]

# Assemble feature columns into a single vector required by PySpark ML
assembler = VectorAssembler(inputCols=feature_names, outputCol="features")
ICLRdata_assembled = assembler.transform(ICLRdata_labelled)

# Testing for multicollinearity
pearson_matrix = Correlation.corr(ICLRdata_assembled, "features", "pearson").collect()[0][0]

print("Pearson Correlation Matrix:")
print(f"{'':>15}", end="")
for name in feature_names:
    print(f"{name:>15}", end="")
print()
for i, row_name in enumerate(feature_names):
    print(f"{row_name:>15}", end="")
    for j in range(len(feature_names)):
        print(f"{pearson_matrix[i, j]:>15.3f}", end="")
    print()

# Stratified split: each class split separately to preserve class proportions
class_not_accepted = ICLRdata_assembled.filter(F.col("label") == 0.0)
class_accepted     = ICLRdata_assembled.filter(F.col("label") == 1.0)
train_not, test_not = class_not_accepted.randomSplit([0.8, 0.2], seed=42)
train_acc, test_acc = class_accepted.randomSplit([0.8, 0.2], seed=42)
train = train_not.union(train_acc)
test  = test_not.union(test_acc)
```

```
print("\nTraining set class distribution:")
train.groupBy("label").count().orderBy("label").show()
print("Test set class distribution:")
test.groupBy("label").count().orderBy("label").show()

binary_evaluator = BinaryClassificationEvaluator(
    labelCol="label", rawPredictionCol="rawPrediction", metricName="areaUnderROC"
)

# Train Random Forest with feature importances to show which score statistics
# drive acceptance decisions. CrossValidator tunes maxDepth and numTrees to balance
# expressivity against overfitting
rf = RandomForestClassifier(
    featuresCol="features", labelCol="label",
    numTrees=200, maxDepth=10, minInstancesPerNode=5,
    featureSubsetStrategy="sqrt", seed=42
)
rf_paramGrid = ParamGridBuilder() \
    .addGrid(rf.maxDepth, [5, 10]) \
    .addGrid(rf.numTrees, [100]) \
    .build()
rf_cv = CrossValidator(
    estimator=rf, estimatorParamMaps=rf_paramGrid,
    evaluator=binary_evaluator, numFolds=3, seed=42, parallelism=4
)
print("\nTraining Random Forest with Cross Validation...")
rf_cv_model   = rf_cv.fit(train)
rf_best_model = rf_cv_model.bestModel
print("Best maxDepth:", rf_best_model.getOrDefault("maxDepth"))
print("Best numTrees:", rf_best_model.getOrDefault("numTrees"))

# Print feature importances to show the relative contribution of each score feature
print("\nRandom Forest Feature Importances:")
for label, importance in zip(feature_labels, rf_best_model.featureImportances):
    bar = "" * int(importance * 50)
    print(f"{label:<52}: {importance:.4f} {bar}")

# Generate predictions on the held-out test set
rf_predictions = rf_best_model.transform(test)
```

```python
# Compute accuracy and AUC
rf_accuracy = rf_predictions.filter(
    F.col("prediction") == F.col("label")
).count() / rf_predictions.count()

rf_auc = binary_evaluator.evaluate(rf_predictions)

# Print confusion matrix
print("\nRandom Forest Confusion Matrix:")
print(f"{'':>25} {'Predicted: Not Accepted':>25} {'Predicted: Accepted':>20}")
cm = rf_predictions.groupBy("label") \
    .pivot("prediction", [0.0, 1.0]) \
    .count() \
    .fillna(0) \
    .orderBy("label")
class_names = ["Actual: Not Accepted", "Actual: Accepted"]
for row in cm.collect():
    label  = int(row["label"])
    pred_0 = int(row["0.0"] or 0)
    pred_1 = int(row["1.0"] or 0)
    print(f"{class_names[label]:>25} {pred_0:>25} {pred_1:>20}")

# Print final evaluation summary
print("\n" + "=" * 45)
print("RANDOM FOREST EVALUATION RESULTS")
print("=" * 45)
print(f"{'Metric':<25} {'Random Forest':>15}")
print("-" * 45)
print(f"{'Accuracy':<25} {rf_accuracy:>15.4f}")
print(f"{'AUC':<25} {rf_auc:>15.4f}")
print("=" * 45)
```

Extension Results:

The random forest model performed strongly on unseen data, achieving an AUC of 0.962 and accuracy of 90%. The feature importance scores show that mean score and minimum score together account for 92.5% of the model's predictive power. Panel size and reviewer disagreement contribute almost nothing. The model misclassified 17% of accepted papers as rejected and 6% of rejected papers as accepted.

```
+----------------+-------------+-------------+-----------------+-----------------------------------------------------+
|      Mean Score|Minimum Score|Maximum Score|Number of Reviewers|Standard Deviation of Scores (Reviewer Disagreement)|
+----------------+-------------+-------------+-----------------+-----------------------------------------------------+
|             5.0|          4.0|          6.0|                3|                                0.816496580927726|
|6.333333333333333|          6.0|          7.0|                3|                                0.471404520791031|
|             7.0|          6.0|          8.0|                3|                                0.816496580927726|
|5.666666666666667|          5.0|          6.0|                3|                                0.471404520791031|
|             7.0|          5.0|          9.0|                3|                                 1.632993161855452|
+----------------+-------------+-------------+-----------------+-----------------------------------------------------+
only showing top 5 rows
Pearson Correlation Matrix:
                mean_score  min_score  max_score  num_reviewers  std_score
    mean_score       1.000      0.850      0.839          0.050     -0.045
     min_score       0.850      1.000      0.545         -0.044     -0.494
     max_score       0.839      0.545      1.000          0.137      0.438
 num_reviewers       0.050     -0.044      0.137          1.000      0.118
     std_score      -0.045     -0.494      0.438          0.118      1.000
```

e 6).

```
Training set class distribution:
+-----+-----+
|label|count|
+-----+-----+
|  0.0|19438|
|  1.0| 9151|
+-----+-----+

Test set class distribution:
+-----+-----+
|label|count|
+-----+-----+
|  0.0| 4830|
|  1.0| 2198|
+-----+-----+
```

```
Training Random Forest with Cross Validation...
Best maxDepth: 10
Best numTrees: 100

Random Forest Feature Importances:
Mean Score                                        : 0.6859
Minimum Score                                     : 0.2389
Maximum Score                                     : 0.0579
Number of Reviewers                               : 0.0020
Standard Deviation of Scores (Reviewer Disagreement): 0.0152

Random Forest Confusion Matrix:
                        Predicted: Not Accepted  Predicted: Accepted
    Actual: Not Accepted                    4532                 298
        Actual: Accepted                     383                1815
```

```
============================================
RANDOM FOREST EVALUATION RESULTS
============================================
Metric                          Random Forest
--------------------------------------------
Accuracy                               0.9031
AUC                                    0.9618
============================================
```

Extension Discussion:

The extension fulfilled its aim. The model's strong performance confirms that acceptance decisions are based on merit rather than driven by other factors. This is reassuring

for the committee. The feature importance finding adds a practical insight: what matters is not whether all reviewers agree, but whether the average and lowest individual scores are high. The committee therefore does not need to standardise panel sizes to improve decision consistency. The 17% of accepted papers misclassified as rejected represent cases where scores alone cannot explain the decision outcome. Reviewer discussion or author rebuttal could have played a role.

The ICLR committee should apply sentiment analysis to rebuttal and discussion threads on OpenReview to identify which rebuttal strategies are most common among papers that achieved acceptance despite their mixed scores. The committee could then use these findings to provide clearer guidance to authors on how to respond effectively to mixed reviews.

Question 2: Accepted Papers with Low Scores and Reviewer Disagreement Analysis

Assumptions:

Single low score as reviewer reservation: The question counts any paper where at least one reviewer scored it below 5, regardless of how many reviewers the paper had. Every score below 5 is assumed to reflect a genuine negative assessment. If a low score was submitted in error, that paper would still be flagged, overstating the count.

Panel size does not significantly distort the count: Papers with larger review panels have a greater chance of containing at least one score below 5 by chance alone. It is assumed this effect is not large enough to significantly distort the count of 1,596, though this may not fully hold for papers with unusually large panels.

Implementation:

ICLRdata_clean was used as both scores and decisions are required. F.exists() checked whether any score in the array was below 5, chosen over explode() to avoid row duplication.

```
▶ ✓ 11 minutes ago (1s)                                                    5

# ============================================================================
# QUESTION 2 – NUMBER OF PAPERS ACCEPTED WITH AT LEAST ONE SCORE UNDER 5
# ============================================================================

# Filter to accepted papers only using the same regex as Q1
df_accepted = ICLRdata_clean.filter(
    F.col("decision").rlike("(?i)accept|oral|poster|spotlight|workshop|talk|notable")
)

# Count accepted papers where at least one score in the array is below 5
q2_count = df_accepted.filter(
    F.exists(F.col("scores"), lambda x: x < 5)
).count()

total_accepted = df_accepted.count()
print(f"Total accepted papers: {total_accepted}")
print(f"Q2 Answer: {q2_count} accepted papers had at least one score under 5")
```

Results:

```
> df_accepted:  pyspark.sql.connect.dataframe.DataFrame = [year: long, id: string ... 8 more fields]
Total accepted papers: 11349
Q2 Answer: 1596 accepted papers had at least one score under 5
```

Discussion:

Roughly 1 in 7 accepted papers (1,596 out of 11,349) had at least one reviewer who scores it below 5. This is evidence that the review process is working as intended. A process with zero such cases would be a concern, as it would signal that reviewers were not making their own independent judgements.

The core result established the count but did not explain what drives it. The committee needs to know whether these low scores reflect genuine reviewer disagreement or if they are scoring errors. Although Question 1's extension found that reviewer disagreement has almost no influence on whether a paper is accepted, this extension asks a more specific question: among accepted papers, does reviewer disagreement explain why some contained at least one low score?

Extension Assumptions:

Standard deviation reliably captures reviewer disagreement: Score standard deviation is assumed to measure genuine differences of opinion between reviewers. This holds for panels with three or more reviewers, where a spread of scores meaningfully reflects disagreement. For papers with only two reviewers, standard deviation is more sensitive to a single divergent score and may overstate disagreement. Since the majority of papers in the dataset have three or more reviewers, this assumption holds for most cases.

Extension Implementation:

Score mean and standard deviation were calculated using F.aggregate() and F.sqrt(). approxQuantile() calculated thresholds at the 33rd and 66th percentiles, assigning papers to low, medium, or high disagreement tiers. Three aggregations produced acceptance rate by tier, accepted papers with at least one low score by tier, and disagreement trends over time.

```
#=====================================================================
# QUESTION 2 EXTENSION: REVIEWER DISAGREEMENT ANALYSIS
#=====================================================================
# ── COMPUTE SCORE STANDARD DEVIATION PER PAPER ──────────────────────
# mean_score is needed first so std dev can reference it in the same aggregate
df_with_stats = ICLRdata_clean \
    .withColumn("mean_score",
        F.aggregate(F.col("scores"), F.lit(0.0),
            lambda acc, x: acc + x.cast("double")
        ) / F.size(F.col("scores"))
    ) \
    .withColumn("score_std",
        F.sqrt(
            F.aggregate(F.col("scores"), F.lit(0.0),
                lambda acc, x: acc + (x.cast("double") - F.col("mean_score")) ** 2
            ) / F.size(F.col("scores"))
        )
    ) \
    .withColumn("accepted",
        F.when(F.col("decision").rlike(
            "(?i)accept|oral|poster|spotlight|workshop|talk|notable"), 1.0
        ).otherwise(0.0)
    )
```

```
# ── BIN INTO DISAGREEMENT TIERS ─────────────────────────────────────
# Use approxQuantile to find the 33rd and 66th percentile thresholds
p33, p66 = df_with_stats.approxQuantile("score_std", [0.33, 0.66], 0.001)
print(f"Disagreement thresholds — Low: Score Standard Deviation < {p33:.3f}, Medium: Score Standard
Deviation {p33:.3f}–{p66:.3f}, High: Score Standard Deviation > {p66:.3f}")

df_tiered = df_with_stats.withColumn("disagreement_tier",
    F.when(F.col("score_std") <= p33, "Low")
     .when(F.col("score_std") <= p66, "Medium")
     .otherwise("High")
)

# ── ACCEPTANCE RATE BY DISAGREEMENT TIER ────────────────────────────
print("\nAcceptance rate by reviewer disagreement tier:")
df_tiered \
    .groupBy("disagreement_tier") \
    .agg(
        F.count("id").alias("Total Paper Count in Tier"),
        F.round(F.avg("score_std"), 3).alias("Score Standard Deviation in Tier"),
        F.round(F.avg("accepted") * 100, 1).alias("Acceptance Rate in Tier")
    ) \
    .orderBy("disagreement_tier") \
    .toDF("Disagreement Tier", "Total Paper Count in Tier", "Score Standard Deviation in Tier",
    "Acceptance Rate in Tier (%)") \
    .show(truncate=False)
```

```python
# ── DISAGREEMENT TIER FOR PAPERS WITH AT LEAST ONE LOW SCORE ──────────────
# Cross tabulate the Q2 result with disagreement tier. Do papers with a low
# score tend to come from high disagreement reviews or low disagreement ones?
print("\nAccepted papers by reviewer disagreement tier and whether they contain a low score:")
df_tiered \
    .filter(F.col("accepted") == 1.0) \
    .withColumn("has_low_score",
        F.when(F.exists(F.col("scores"), lambda x: x < 5), "Yes").otherwise("No")
    ) \
    .groupBy("disagreement_tier", "has_low_score") \
    .agg(F.count("id").alias("paper_count")) \
    .orderBy("disagreement_tier", "has_low_score") \
    .toDF("Disagreement Tier", "Has at Least One Low Score", "Number of Accepted Papers") \
    .show(truncate=False)

# ── DISAGREEMENT TREND OVER TIME ──────────────────────────────────────────
# Score standard deviation and Average score of accepted papers per year
print("\nReviewer disagreement (score standard deviation) and average score on accepted papers by year:")
df_tiered \
    .filter(F.col("accepted") == 1.0) \
    .groupBy("year") \
    .agg(
        F.count("id").alias("accepted_papers"),
        F.round(F.avg("score_std"), 3).alias("avg_score_std"),
        F.round(F.avg("mean_score"), 3).alias("avg_mean_score")
    ) \
    .orderBy("year") \
    .toDF("Year", "Number of Accepted Papers", "Score Standard Deviation for Accepted Papers (Reviewer Disagreement)", "Average Score for Accepted Papers") \
    .show(truncate=False)
```

Extension Results:

```
Disagreement thresholds — Low: Score Standard Deviation < 0.866, Medium: Score Standard Deviation 0.866–1.225, High: Score Standard Deviation > 1.225

Acceptance rate by reviewer disagreement tier:
+----------------+-----------------------+-----------------------------+-------------------------+
|Disagreement Tier|Total Paper Count in Tier|Score Standard Deviation in Tier|Acceptance Rate in Tier (%)|
+----------------+-----------------------+-----------------------------+-------------------------+
|High            |11334                  |1.628                        |25.4                     |
|Low             |14155                  |0.476                        |35.5                     |
|Medium          |10128                  |1.04                         |34.1                     |
+----------------+-----------------------+-----------------------------+-------------------------+
```

```
Accepted papers by reviewer disagreement tier and whether they contain a low score:
+----------------+---------------------------+--------------------------+
|Disagreement Tier|Has at Least One Low Score|Number of Accepted Papers|
+----------------+---------------------------+--------------------------+
|High            |No                         |1456                      |
|High            |Yes                        |1418                      |
|Low             |No                         |4984                      |
|Low             |Yes                        |40                        |
|Medium          |No                         |3313                      |
|Medium          |Yes                        |138                       |
+----------------+---------------------------+--------------------------+
```

```
Reviewer disagreement (score standard deviation) and average score on accepted papers by year:
+----+-----------------------+-----------------------------------------------------------------------+-----------------------------+
|Year|Number of Accepted Papers|Score Standard Deviation for Accepted Papers (Reviewer Disagreement)|Average Score for Accepted Papers|
+----+-----------------------+-----------------------------------------------------------------------+-----------------------------+
|2017|245                    |0.75                                                                   |6.635                        |
|2018|425                    |0.829                                                                  |6.321                        |
|2019|502                    |0.728                                                                  |6.613                        |
|2020|687                    |0.986                                                                  |6.243                        |
|2021|859                    |0.806                                                                  |6.524                        |
|2022|1094                   |0.963                                                                  |6.628                        |
|2023|1573                   |0.988                                                                  |6.612                        |
|2024|2261                   |0.958                                                                  |6.463                        |
|2025|3703                   |0.939                                                                  |6.457                        |
+----+-----------------------+-----------------------------------------------------------------------+-----------------------------+
```

Extension Discussion:

The extension succeeded in its aim. The second table reveals how these 1,596 papers came about. In panels with low disagreement, low scores were extremely rare (40 out of 5,024 accepted papers had at least one). But in panels where reviewers strongly disagreed, nearly half of the accepted papers had at least one low score (1,418 out of 2,874 accepted papers). This suggests that reviewer disagreement plays a significant role in explaining why some accepted papers contained at least one low score, rather than random scoring errors.

Papers with high reviewer disagreement had a lower acceptance rate (25.4%) than papers where reviewers mostly agreed (35.5%). This is consistent with a review process that is working as it should.

The last table shows that reviewer disagreement on accepted papers has increased over the years, even though average scores have remained stable. The score threshold has not dropped, but consensus has softened slightly. The committee should investigate whether this reflects intentional policy change or strain from rising submission volumes.

A useful next step would be to identify which research areas show the highest reviewer disagreement. The ICLR committee could identify these research areas by grouping papers by keywords and calculating average reviewer disagreement by topic. Papers in research areas with high reviewer disagreement should be assigned additional specialist reviewers, as this is likely to reflect a lack of topic expertise rather than paper quality differences.

Question 3: Top 10 Papers by Mean Reviewer Score and Score Inflation Analysis

Assumptions:

Mean score accurately reflects actual reviewer assessments: Each paper's mean score is calculated directly from the scores array. Every score in the array is assumed to have been submitted intentionally and recorded correctly. Papers with empty score arrays are excluded. If any scores were recorded in error, the rankings could change. This is especially significant for papers near the boundary of the top 10, where small differences in mean score determine inclusion.

Tied papers are ordered consistently but arbitrarily: Five of the ten papers share a mean score of exactly 9.0. Tied papers are then ordered by year and ID, but this ordering is arbitrary. Any other paper in the dataset with a mean score of 9.0 could have appeared in the top 10 instead, depending on its year and ID. The committee's interest is in identifying high-scoring papers rather than their precise order.

Implementation:

ICLRdata was filtered to exclude empty score arrays. The mean score was computed using F.aggregate(), avoiding the need to unpack the array into separate rows. Results were ordered by mean score descending, then year and ID for tiebreakers, before .limit(10) returned the top 10 papers.

```
▶ ✓ 01:29 PM (1s)                                                    7

# ============================================================
# QUESTION 3 — TOP 10 PAPERS BY HIGHEST MEAN SCORE
# ============================================================
# Uses ICLRdata filtered to non empty scores only. No decision required.
# This avoids excluding valid submissions that lack a recorded outcome.

# Compute mean score per paper from the scores array
df_q3_base = ICLRdata.filter(F.size(F.col("scores")) > 0) \
    .withColumn("mean_score",
        F.aggregate(
            F.col("scores"), F.lit(0.0),
            lambda acc, x: acc + x.cast("double")
        ) / F.size(F.col("scores"))
    )

# Select the top 10 papers by mean score
df_q3 = df_q3_base \
    .select("id", "mean_score", "year") \
    .orderBy(F.col("mean_score").desc(), F.col("year").desc(), F.col("id").asc()) \
    .limit(10)

print("Q3 Answer — Top 10 papers by highest mean score:")
df_q3.show(truncate=False)
```

Results:

```
Q3 Answer — Top 10 papers by highest mean score:
+----------+-----------------+----+
|id        |mean_score       |year|
+----------+-----------------+----+
|u1cQYxRI1H|10.0             |2025|
|Sy8gdB9xx |9.666666666666666|2017|
|6Mxhg9PtDE|9.5              |2025|
|88nT0j5jAn|9.333333333333334|2023|
|LyJi5ugyJx|9.2              |2025|
|DJSZGGZYVi|9.0              |2025|
|Ha6RTeWMd0|9.0              |2025|
|WCRQFlji2q|9.0              |2025|
|YrycTjllL0|9.0              |2025|
|gc8QAQfXv6|9.0              |2025|
+----------+-----------------+----+
```

Discussion:

The highest ranked paper received a perfect average score of 10. This means that every reviewer gave it the highest score possible. This is very rare across almost 56,000 submissions and represents a high-quality piece of work that should be examined by future authors aspiring for their papers to be accepted into the ICLR.

Eight of the top 10 papers are from 2025. This concentration is likely a reflection of the dataset rather than a genuine quality shift. 2025 is one of the largest years for submissions in the dataset (11,663 papers), meaning there are simply more candidates from that year from which high-scoring papers can emerge. The committee should not interpret this as evidence that 2025 submissions were of higher quality than those from earlier years.

If scoring standards have shifted over time, year-over-year comparisons would be unreliable and any findings in this report that rely on scores would be called into question. To investigate whether score inflation is present, a linear regression model was trained to test whether submission year predicts mean score.

Extension Assumption:

Score inflation would show up as a gradual upward trend over time: This extension tests for score inflation by checking whether scores have risen consistently year on year. If scores instead jumped suddenly in one year or went up and then back down, this test would not fully detect it. However, since the results show that year has almost no influence on scores in any direction, the conclusion that there is no score inflation holds regardless.

Extension Implementation:

Three subsets were created (all papers, accepted only, and rejected only) to test whether inflation patterns differed by outcome group. A linear regression model was fitted on each using an 80/20 split, with R^2 as the primary metric as it directly measures year's explanatory power.

4 minutes ago (12s) 8

```
# ============================================================
# QUESTION 3 EXTENSION — SCORE INFLATION ANALYSIS USING
# LINEAR REGRESSION
# ============================================================

# Prepare three subsets: all papers, accepted only, rejected only
df_all          = df_q3_base.select("year", "mean_score")
df_q3_known     = df_q3_base.filter(F.col("decision") != "")
df_accepted_reg = df_q3_known \
    .filter(F.col("decision").rlike(
        "(?i)accept|oral|poster|spotlight|workshop|talk|notable")) \
    .select("year", "mean_score")
df_rejected_reg = df_q3_known \
    .filter(~F.col("decision").rlike(
        "(?i)accept|oral|poster|spotlight|workshop|talk|notable")) \
    .select("year", "mean_score")

print(f"All papers:      {df_all.count()} rows")
print(f"Accepted papers: {df_accepted_reg.count()} rows")
print(f"Rejected papers: {df_rejected_reg.count()} rows")

# Wrap year into a feature vector and apply an 80/20 train/test split to each subset
assembler_reg = VectorAssembler(inputCols=["year"], outputCol="features")
df_all_assembled      = assembler_reg.transform(df_all)
df_accepted_assembled = assembler_reg.transform(df_accepted_reg)
df_rejected_assembled = assembler_reg.transform(df_rejected_reg)

train_all,      test_all      = df_all_assembled.randomSplit([0.8, 0.2], seed=42)
train_accepted, test_accepted = df_accepted_assembled.randomSplit([0.8, 0.2], seed=42)
train_rejected, test_rejected = df_rejected_assembled.randomSplit([0.8, 0.2], seed=42)

print("\nTrain/test split sizes:")
print(f"All papers      — Train: {train_all.count()}, Test: {test_all.count()}")
print(f"Accepted papers — Train: {train_accepted.count()}, Test: {test_accepted.count()}")
print(f"Rejected papers — Train: {train_rejected.count()}, Test: {test_rejected.count()}")
```

```python
# Fit a separate linear regression on each training set
lr_reg = LinearRegression(featuresCol="features", labelCol="mean_score")
lr_model_all      = lr_reg.fit(train_all)
lr_model_accepted = lr_reg.fit(train_accepted)
lr_model_rejected = lr_reg.fit(train_rejected)

# Print slope and intercept for each group. Slope shows score change per year
print("\n" + "=" * 58)
print("LINEAR REGRESSION COEFFICIENTS")
print("=" * 58)
print(f"{'Group':<20} {'Slope (per year)':>18} {'Intercept':>15}")
print("-" * 58)
print(f"{'All papers':<20} "
      f"{lr_model_all.coefficients[0]:>18.4f} "
      f"{lr_model_all.intercept:>15.4f}")
print(f"{'Accepted papers':<20} "
      f"{lr_model_accepted.coefficients[0]:>18.4f} "
      f"{lr_model_accepted.intercept:>15.4f}")
print(f"{'Rejected papers':<20} "
      f"{lr_model_rejected.coefficients[0]:>18.4f} "
      f"{lr_model_rejected.intercept:>15.4f}")
print("=" * 58)

# Generate predictions on each held out test set
predictions_all      = lr_model_all.transform(test_all)
predictions_accepted = lr_model_accepted.transform(test_accepted)
predictions_rejected = lr_model_rejected.transform(test_rejected)

# Evaluate each model with R² as the primary metric
r2_evaluator = RegressionEvaluator(
    labelCol="mean_score", predictionCol="prediction", metricName="r2"
)

r2_all      = r2_evaluator.evaluate(predictions_all)
r2_accepted = r2_evaluator.evaluate(predictions_accepted)
r2_rejected = r2_evaluator.evaluate(predictions_rejected)
```

```python
# Print R² results for each group
print("\n" + "=" * 58)
print("MODEL EVALUATION RESULTS (TEST SET)")
print("=" * 58)
print(f"{'Group':<20} {'R²':>18}")
print("-" * 58)
print(f"{'All papers':<20} {r2_all:>18.4f}")
print(f"{'Accepted papers':<20} {r2_accepted:>18.4f}")
print(f"{'Rejected papers':<20} {r2_rejected:>18.4f}")
print("=" * 58)
```

Extension Results:

```
All papers:      35617 rows
Accepted papers: 11349 rows
Rejected papers: 24268 rows

Train/test split sizes:
All papers      — Train: 28507, Test: 7110
Accepted papers — Train: 9151, Test: 2198
Rejected papers — Train: 19438, Test: 4830

============================================================
LINEAR REGRESSION COEFFICIENTS
============================================================
Group                   Slope (per year)       Intercept
------------------------------------------------------------
All papers                        0.0128        -20.8712
Accepted papers                  -0.0003          7.1638
Rejected papers                   0.0321        -60.4755
============================================================

============================================================
MODEL EVALUATION RESULTS (TEST SET)
============================================================
Group                                  R²
------------------------------------------------------------
All papers                         0.0006
Accepted papers                   -0.0001
Rejected papers                    0.0059
============================================================
```

Extension Discussion:

The extension succeeded in its aim. The results show no evidence of score inflation across any of the three outcome groups. R^2 values are near zero across all 3 of the decision groups. This means that year explains almost none of the variation in scores. The slope for the accepted papers in particular was almost flat at -.0003, meaning accepted paper scores have remained the most stable over the years. Scoring standards have remained consistent despite ICLR's significant growth. This means that scores from 2017 are directly comparable to scores from 2025.

A useful next step would be to compare the average panel size of the top 10 papers against the dataset average. Unlike Question 1's extension which tested panel size's influence on acceptance decisions, this would test whether panel size influences mean score directly. If

top-scoring papers tend to have smaller panels, panel size may be influencing the rankings. The ICLR committee should then consider setting a minimum panel size to ensure rankings reflect paper quality.

Question 4: Keyword Spikes and Acceptance Rate Trajectories

Assumptions:

Each keyword occurrence represents a distinct research focus: Keywords are counted per submission per year, meaning a keyword that appears in 87 papers is assumed to reflect 87 independent research contributions to that topic. This assumption holds for most keywords but broad keywords may group together papers with very different research questions. For example, "LLM" may appear across papers with very different research questions, meaning the spike reflects terminology implementation rather than a single concentrated research direction.

The spike criterion identifies genuine shifts in research focus rather than noise: A threshold of fewer than 5 occurrences one year and more than 50 the next is assumed to separate meaningful topic emergence from natural year-to-year variation. This assumption holds given the scale of the dataset. A keyword appearing fewer than 5 times across thousands of submissions is genuinely rare, and crossing 50 in a single year represents a significant change rather than gradual growth.

Implementation:

ICLRdata was filtered to exclude empty keyword arrays. F.explode() produced one row per keyword per paper. Three normalisation steps were applied: F.lower(), F.trim(), and F.regexp_replace() to collapse internal whitespace. A self-join identified consecutive year pairs and the spike filter (count_prev < 5 and count_next > 50) was applied, with results ordered by spike year count descending.

```
#=======================================================
#         QUESTION 4 — KEYWORDS THAT SPIKED <5 TO >50
#=======================================================
# Uses ICLRdata (all submissions).
# Keywords normalised to lowercase with collapsed internal whitespace.

# Explode and normalise keywords. One row per keyword per paper
keywords_exploded = ICLRdata \
    .filter(F.size(F.col("keywords")) > 0) \
    .select("year", F.explode(F.col("keywords")).alias("keyword")) \
    .withColumn("keyword", F.lower(F.col("keyword"))) \
    .withColumn("keyword", F.trim(F.col("keyword"))) \
    .withColumn("keyword", F.regexp_replace(F.col("keyword"), "\\s+", " ")) \
    .filter(F.col("keyword") != "")

# Count keyword occurrences per year
kw_counts = keywords_exploded \
    .groupBy("year", "keyword") \
    .count() \
    .withColumnRenamed("count", "kw_count")

# Self join on keyword and consecutive year to find year-on-year pairs
kw_prev = kw_counts \
    .withColumnRenamed("year", "year_prev") \
    .withColumnRenamed("kw_count", "count_prev")
kw_next = kw_counts \
    .withColumnRenamed("year", "year_next") \
    .withColumnRenamed("kw_count", "count_next")

kw_joined = kw_prev.join(
    kw_next,
    on=(kw_prev["keyword"] == kw_next["keyword"]) &
       (kw_prev["year_prev"] + 1 == kw_next["year_next"])
).select(
    kw_prev["keyword"],
    "year_prev", "count_prev",
    "year_next", "count_next"
)
```

```
# Apply the spike filter: fewer than 5 one year, more than 50 the next
kw_q4 = kw_joined.filter(
    (F.col("count_prev") < 5) & (F.col("count_next") > 50)
)

print("Q4 Answer — Keywords that spiked from <5 to >50:")
kw_q4.orderBy(F.col("count_next").desc()) \
    .toDF("Keyword", "Pre-Spike Year", "Paper Count in Pre-Spike Year", "Spike Year", "Paper Count in
    Spike Year") \
    .show(10, truncate=False)
```

Results:

```
Q4 Answer — Keywords that spiked from <5 to >50:
+----------------------+-------------+----------------------------+----------+-------------------------+
|Keyword               |Pre-Spike Year|Paper Count in Pre-Spike Year|Spike Year|Paper Count in Spike Year|
+----------------------+-------------+----------------------------+----------+-------------------------+
|test-time scaling     |2025          |3                           |2026      |87                       |
|llm                   |2023          |4                           |2024      |76                       |
|large reasoning models|2025          |1                           |2026      |57                       |
|spatial reasoning     |2025          |3                           |2026      |57                       |
+----------------------+-------------+----------------------------+----------+-------------------------+
```

Discussion:

Only four keywords met the spike criterion across almost ten years and over 55,000 submissions. This means that the topics at ICLR are mostly stable from year to year. However, not all spikes reflect new research. ‘LLM’ likely represents a terminology change from ‘large language models’, suggesting the topic predates the spike.

The three 2026 spikes reflect a new shift towards making models reason and think more effectively. Therefore, the committee should consider whether current reviewer panels have the expertise required to assess these new topics fairly.

The core result identified which keywords spiked but did not reveal whether paper quality changed. The committee needs to know whether topic popularity attracts weaker submissions and whether quality recovers as topics mature. To investigate this, acceptance rates were tracked before, during, and after each spike.

Extension Assumption:

Normalisation produces consistent keyword matching across years: Keywords are converted to lowercase, trimmed of leading and trailing whitespace, and internal whitespace is collapsed to a single space before counting. This assumes that matching keywords (but just written with different capitalisation or spacing) should be counted as the same keyword. This assumption holds for the majority of cases. However, it means that "LLM" and "large language models" are still counted separately despite referring to the same concept. This may cause the spike for "LLM" to understate the true growth of that research area.

Extension Implementation:

Where a keyword spiked more than once, only the earliest spike was retained. Acceptance was encoded using the same .rlike() pattern as Question 1 and each year classified as pre-spike, spike year, or post-spike using F.when().

```
#==============================================================================
# QUESTION 4 — EXTENSION: KEYWORD ACCEPTANCE RATE BEFORE AND AFTER THE SPIKE
#==============================================================================

# ── PREPARE SPIKING KEYWORD TABLE ──────────────────────────────────────────

# Keep only the first spike per keyword; extract pre spike and spike years
w_first_spike = Window.partitionBy("keyword").orderBy("year_next")
spiking_kws = kw_q4 \
    .withColumn("rank", F.rank().over(w_first_spike)) \
    .filter(F.col("rank") == 1) \
    .select(
        "keyword",
        F.col("year_prev").alias("pre_spike_year"),
        F.col("year_next").alias("spike_year"),
        F.col("count_prev").alias("count_pre"),
        F.col("count_next").alias("count_spike")
    )

# ── BUILD KEYWORD-PAPER ACCEPTANCE TABLE ──────────────────────────────────

# Explode and normalise keywords. Same cleaning as Q4 core so join keys match
kw_paper_acceptance = ICLRdata \
    .filter(F.col("decision") != "") \
    .withColumn("accepted",
        F.when(F.col("decision").rlike(
            "(?i)accept|oral|poster|spotlight|workshop|talk|notable"), 1.0
        ).otherwise(0.0)
    ) \
    .filter(F.size(F.col("keywords")) > 0) \
    .select(
        "year", "accepted",
        F.explode(F.col("keywords")).alias("keyword")
    ) \
    .withColumn("keyword", F.lower(F.col("keyword"))) \
    .withColumn("keyword", F.trim(F.col("keyword"))) \
    .withColumn("keyword", F.regexp_replace(F.col("keyword"), "\\s+", " ")) \
    .filter(F.col("keyword") != "")
```

```
# ── YEAR-BY-YEAR TRAJECTORY ────────────────────────────────────────────────

# Full year-by-year acceptance rate per spiking keyword. Reveals whether
# any shift was gradual or sudden
kw_yearly_acceptance = kw_paper_acceptance \
    .join(spiking_kws.select("keyword", "spike_year"), on="keyword", how="inner") \
    .groupBy("keyword", "spike_year", "year") \
    .agg(
        F.round(F.avg("accepted") * 100, 1).alias("acceptance_rate_percentage"),
        F.count("accepted").alias("paper_count")
    ) \
    .withColumn("period",
        F.when(F.col("year") < F.col("spike_year"),  "Pre-spike")
         .when(F.col("year") == F.col("spike_year"), "Spike year")
         .otherwise("Post-spike")
    ) \
    .orderBy("keyword", "year")

print("Year-by-year acceptance rate trajectory for each spiking keyword:")
kw_yearly_acceptance.select(
    "keyword", "year", "period", "paper_count", "acceptance_rate_percentage"
).show(50, truncate=False)
```

Extension Results:

```
Year-by-year acceptance rate trajectory for each spiking keyword:
+----------------------+----+----------+-----------+--------------------------+
|keyword               |year|period    |paper_count|acceptance_rate_percentage|
+----------------------+----+----------+-----------+--------------------------+
|large reasoning models|2025|Pre-spike |1          |0.0                       |
|large reasoning models|2026|Spike year|1          |0.0                       |
|llm                   |2023|Pre-spike |4          |50.0                      |
|llm                   |2024|Spike year|76         |25.0                      |
|llm                   |2025|Post-spike|350        |29.7                      |
|llm                   |2026|Post-spike|6          |0.0                       |
|spatial reasoning     |2024|Pre-spike |1          |0.0                       |
|spatial reasoning     |2025|Pre-spike |3          |33.3                      |
|test-time scaling     |2025|Pre-spike |3          |66.7                      |
|test-time scaling     |2026|Spike year|1          |0.0                       |
+----------------------+----+----------+-----------+--------------------------+
```

Extension Discussion:

The extension reveals that when “LLM” spiked in 2024, the acceptance rate dropped from 50% to 25%. This suggests that topic popularity attracts varying levels of submission quality. However, the acceptance rate recovered to 30% by 2025, indicating that quality improves as research topics mature. The 2026 acceptance rates should be treated with caution as most 2026 submissions were unreviewed at the time that this dataset was created.

A useful next step would be to monitor acceptance rates for the three 2026 keywords over several future conference cycles to determine if quality improves as the topics mature, or whether the ICLR committee needs to provide clearer guidance to both reviewers and authors on what is expected for these new topics.

Question 5: OpenAlex Coverage and Geographic Distribution of ICLR Submissions

Assumptions:

Title matching reliably identifies the same paper across both datasets: The two datasets share no common identifier, so papers are matched on title string alone. Titles are assumed to be recorded consistently across both datasets. However, this assumption does not hold. Minor differences such as changes in punctuation, white space, capitalisation, or slight rewording would cause genuine matches to be missed, understating the true number of ICLR

papers with OpenAlex records. To partially address this, the extension utilised normalised titles to maximise matches.

Implementation:

ICLRdata and openalex_raw were inner joined on raw titles, as only papers present in both datasets are relevant. F.count('id') counted matched papers per year, which was then joined to total submissions per year from a separate groupBy to compute a match rate.

2 minutes ago (2s) 11

```
# ============================================================
# QUESTION 5 — ICLR PAPERS WITH OPENALEX DATA, BY YEAR
# ============================================================
# All 2,245 OpenAlex records are used. Every record is a potential ICLR match.

# Select only the columns needed from OpenAlex
openalex_iclr = openalex_raw \
    .select(
        F.col("title").alias("oa_title"),
        F.col("authorships")
    )

# Join ICLR to OpenAlex on matching title strings
q5_core_joined = ICLRdata.join(
    openalex_iclr,
    ICLRdata.title == openalex_iclr.oa_title,
    how="inner"
)

# Count total matched ICLR papers across all years
total_matched = q5_core_joined.select(
    F.count("id")
).collect()[0][0]

print(f"Total matched papers (ICLR ∩ OpenAlex): {total_matched}")
total_iclr = ICLRdata.count()
match_rate  = round(total_matched / total_iclr * 100, 1)
print(f"Total ICLR submissions:          {total_iclr}")
print(f"Overall match rate:              {match_rate}%")

# Compute matched paper count and match rate per submission year
total_by_year = ICLRdata \
    .groupBy("year") \
    .agg(F.count("id").alias("total_submissions"))
```

```
print("Q5 Answer — matched papers and match rate by year:")
q5_core_joined \
    .groupBy("year") \
    .agg(F.count("id").alias("matched_papers")) \
    .join(total_by_year, on="year", how="left") \
    .withColumn(
        "match_rate_pct",
        F.round(
            F.col("matched_papers") / F.col("total_submissions") * 100, 1
        )
    ) \
    .orderBy("year") \
    .toDF("ICLR Submission Year", "Number of Matched Papers", "Total Number of Papers in ICLR Dataset", "Match Rate (%)") \
    .show(truncate=False)
```

Results:

```
Total matched papers (ICLR ∩ OpenAlex): 1593
Total ICLR submissions:          55906
Overall match rate:              2.8%
Q5 Answer — matched papers and match rate by year:
```

ICLR Submission Year	Number of Matched Papers	Total Number of Papers in ICLR Dataset	Match Rate (%)
2017	154	489	31.5
2018	309	1012	30.5
2019	199	1569	12.7
2020	433	2593	16.7
2021	423	3009	14.1
2022	52	3422	1.5
2023	19	4955	0.4
2024	1	7401	0.0
2025	2	11663	0.0
2026	1	19793	0.0

Discussion:

The low overall match rate of 2.8% is expected. The OpenAlex scrape only contains 2,245 records, compared to 55,906 ICLR submissions in iclr26v1. This means that even if all of the papers in the OpenAlex scrape matched with the ICLR dataset, the match rate would not exceed 4%. Also, the match rate is significantly higher for the years 2017-2021 (14 % to 32%), before dropping to practically zero from 2022 onwards. This is a reflection of the OpenAlex scrape. It does not contain ICLR papers from recent years.

The core result established how many ICLR papers had OpenAlex records, but did not reveal where those papers came from. The ICLR committee needs to know whether ICLR submissions are geographically concentrated, and whether this is accompanied by differences in reviewer scores or acceptance rates that could indicate bias. To investigate this, matched papers were attributed to countries using author institutional affiliations, and acceptance rates and average reviewer scores were computed per country.

Extension Assumption:

Each author's recorded institutional country reflects where their research was conducted: The extension uses country codes from the OpenAlex authorships field to attribute papers to countries. This assumes that the institution listed for each author is where the research was primarily carried out. This may not hold for researchers who completed their work at one institution but were affiliated with another at the time of publication.

Extension Implementation:

For the extension, a separate normalised title join was constructed and filtered to papers with a known decision. A binary accepted flag was added using the same .rlike() pattern as Question 1. Two consecutive F.explode() operations unpacked the nested authorships and countries arrays. Acceptance rate and reviewer score were computed in separate aggregations and left joined, since not all papers have recorded reviewer scores. A minimum threshold of 10 papers per country was applied.

```
# ======================================================================
# QUESTION 5 EXTENSION — GEOGRAPHIC ANALYSIS OF ICLR SUBMISSIONS
# ======================================================================

# Part 1: Country submission volume
# Parts 2 & 3: Acceptance rate and average reviewer score by country,
# combined into a single output.

# ── BUILD EXTENSION JOIN ──────────────────────────────────────────────

# Known decisions only, binary accepted flag added, title normalised for joining
iclr_decisions_titled = ICLRdata \
    .filter(F.col("decision") != "") \
    .withColumn(
        "norm_title",
        F.trim(F.lower(F.col("title")))
    ) \
    .withColumn(
        "accepted",
        F.when(
            F.col("decision").rlike(
                "(?i)accept|oral|poster|spotlight|workshop|talk|notable"
            ),
            1.0
        ).otherwise(0.0)
    )

# Normalise OpenAlex titles for joining
openalex_iclr_norm = openalex_raw \
    .select(
        F.trim(F.lower(F.col("title"))).alias("oa_title"),
        F.col("authorships")
    )

q5_ext_joined = iclr_decisions_titled.join(
    openalex_iclr_norm,
    iclr_decisions_titled.norm_title == openalex_iclr_norm.oa_title,
    how="inner"
)
```

```
total_ext_matched = q5_ext_joined.select(
    F.countDistinct("id")
).collect()[0][0]
print(f"Matched papers with known decision: {total_ext_matched}")

# ISO 3166-1 alpha-2 country code to full name mapping
country_names = {
    "US": "United States", "GB": "United Kingdom", "CN": "China",
    "CA": "Canada", "IL": "Israel", "DE": "Germany", "CH": "Switzerland",
    "FR": "France", "KR": "South Korea", "JP": "Japan", "NL": "Netherlands",
    "SG": "Singapore", "IT": "Italy", "IN": "India", "AU": "Australia",
    "RU": "Russia", "ES": "Spain", "AT": "Austria", "TW": "Taiwan",
    "SE": "Sweden", "BE": "Belgium", "DK": "Denmark", "FI": "Finland",
    "NO": "Norway", "PT": "Portugal", "CZ": "Czech Republic",
    "PL": "Poland", "BR": "Brazil", "IE": "Ireland", "HU": "Hungary",
    "BG": "Bulgaria", "SA": "Saudi Arabia"
}

country_mapping = spark.createDataFrame(
    [(k, v) for k, v in country_names.items()],
    ["country_code", "country_name"]
)

# ── PART 1 — COUNTRY SUBMISSION VOLUME ──────────────────────────────

# Normalise OpenAlex titles for core join used in Part 1
openalex_iclr_norm_core = openalex_raw \
    .select(
        F.trim(F.lower(F.col("title"))).alias("oa_title"),
        F.col("authorships")
    )

# Normalise ICLR titles for core join
iclr_all_titled = ICLRdata \
    .withColumn(
        "norm_title",
        F.trim(F.lower(F.col("title")))
    )
```

```
q5_core_joined_norm = iclr_all_titled.join(
    openalex_iclr_norm_core,
    iclr_all_titled.norm_title == openalex_iclr_norm_core.oa_title,
    how="inner"
)

# Double explode authorships then countries to get one row per country per paper
q5_countries = q5_core_joined_norm \
    .select(
        "id",
        F.explode("authorships").alias("authorship")
    ) \
    .select(
        "id",
        F.explode("authorship.countries").alias("country_code")
    ) \
    .groupBy("country_code") \
    .agg(F.countDistinct("id").alias("paper_count")) \
    .join(country_mapping, on="country_code", how="left") \
    .withColumn("country_name",
        F.coalesce(F.col("country_name"), F.col("country_code"))
    ) \
    .orderBy(F.col("paper_count").desc())

print("Part 1 — Top 10 countries by matched paper count:")
q5_countries.select("country_name", "paper_count") \
    .toDF("Country Name", "Number of Papers") \
    .show(10, truncate=False)
```

```
# ── PART 2 — ACCEPTANCE RATE BY COUNTRY ──────────────────────────────

# Acceptance rate per country; F.avg on a binary accepted column equals the rate
# Minimum 10 papers enforced for statistical reliability
q5_country_acceptance = q5_ext_joined \
    .select(
        "id", "accepted",
        F.explode("authorships").alias("authorship")
    ) \
    .select(
        "id", "accepted",
        F.explode("authorship.countries").alias("country_code")
    ) \
    .groupBy("country_code") \
    .agg(
        F.countDistinct("id").alias("paper_count"),
        F.round(F.avg("accepted"), 3).alias("acceptance_rate")
    ) \
    .filter(F.col("paper_count") >= 10)
```

```python
# ── PART 3 — AVERAGE REVIEWER SCORE BY COUNTRY ──────────────────────────

# Computed separately from Part 2 as it requires filtering to non-empty score
# arrays, which slightly changes the paper count per country. Joined to Part 2
# to produce a single combined table.
q5_country_scores = q5_ext_joined \
    .withColumn("mean_score",
        F.aggregate(
            F.col("scores"), F.lit(0.0),
            lambda acc, x: acc + x.cast("double")
        ) / F.size(F.col("scores"))
    ) \
    .filter(F.size(F.col("scores")) > 0) \
    .select(
        "id", "mean_score",
        F.explode("authorships").alias("authorship")
    ) \
    .select(
        "id", "mean_score",
        F.explode("authorship.countries").alias("country_code")
    ) \
    .groupBy("country_code") \
    .agg(
        F.countDistinct("id").alias("paper_count_scores"),
        F.round(F.avg("mean_score"), 3).alias("avg_score")
    ) \
    .filter(F.col("paper_count_scores") >= 10)
```

```python
# ── COMBINED TABLE — PARTS 2 & 3 JOINED ─────────────────────────────────

# Left join so countries in Part 2 without score data still appear (null avg_score)
# Ordered by paper_count descending to match report table layout
print("Countries by matched paper count with acceptance rate and average reviewer score:")
q5_country_acceptance \
    .join(
        q5_country_scores.select("country_code", "avg_score"),
        on="country_code",
        how="left"
    ) \
    .join(country_mapping, on="country_code", how="left") \
    .withColumn("country_name",
        F.coalesce(F.col("country_name"), F.col("country_code"))
    ) \
    .withColumn("acceptance_rate_pct",
        F.round(F.col("acceptance_rate") * 100, 1)
    ) \
    .select("country_name", "paper_count", "acceptance_rate_pct", "avg_score") \
    .orderBy(F.col("paper_count").desc()) \
    .toDF("Country Name", "Number of Papers", "Acceptance Rate (%)", "Average Reviewer Score") \
    .show(10, truncate=False)
```

Extension Results:

```
Matched papers with known decision: 1601
Part 1 — Top 10 countries by matched paper count:
+--------------+----------------+
|Country Name  |Number of Papers|
+--------------+----------------+
|United States |880             |
|United Kingdom|221             |
|China         |176             |
|Canada        |131             |
|Israel        |107             |
|Germany       |98              |
|Switzerland   |75              |
|France        |52              |
|South Korea   |38              |
|Japan         |37              |
+--------------+----------------+
only showing top 10 rows
```

```
Countries by matched paper count with acceptance rate and average reviewer score:
+--------------+----------------+-------------------+----------------------+
|Country Name  |Number of Papers|Acceptance Rate (%)|Average Reviewer Score|
+--------------+----------------+-------------------+----------------------+
|United States |880             |88.9               |6.281                 |
|United Kingdom|221             |81.0               |6.149                 |
|China         |176             |78.8               |5.984                 |
|Canada        |131             |84.3               |6.008                 |
|Israel        |107             |88.6               |6.279                 |
|Germany       |98              |86.2               |6.183                 |
|Switzerland   |75              |85.5               |6.184                 |
|France        |52              |60.0               |5.624                 |
|South Korea   |38              |90.4               |6.107                 |
|Japan         |37              |72.7               |6.058                 |
+--------------+----------------+-------------------+----------------------+
only showing top 10 rows
```

Extension Discussion:

Both tables show that the United States alone accounted for 880 matched papers. That is a higher matched paper count than the next five countries combined (UK, China, Canada, and Israel). This suggests that researchers at prestigious institutions in the United States have better access to funding, networks, and collaborative opportunities than leading institutions located in other countries. The ICLR committee should consider outreach programs or waiving submission fees to encourage participation from underrepresented countries.

The most reassuring finding for the committee is that average reviewer scores were consistent across the top 10 countries with the highest matched paper counts, falling between 5.98 and 6.28 (with the exception of France). This suggests that papers achieved acceptance

based on merit rather than geographic origin and that the ICLR review process does not suffer from geographic bias.

A valuable next step would be to get institutional location data for more ICLR submissions. This could be done by extracting author affiliation data from OpenReview and adding it to the ICLR dataset. This would help the committee make more informed outreach decisions and track whether US dominance has shifted given the recent rapid expansion of machine learning research in China, India, and South Korea.

Question 6: Low-Scoring ICLR Papers in the 95th Citation Percentile and Citation Analysis for Low-Scoring Accepted Versus Rejected Papers

Assumptions:

Citation count is a reliable measure of a paper's academic value: The question utilises citation count in order to identify papers that the academic community found valuable despite having low average scores. This question assumes that the highly cited papers are positively influential, rather than being highly cited because they are a flawed study. This assumption holds for most papers, although a small number of highly cited papers with low average review scores may be cited as examples of what to avoid.

The citation percentile reflects standing within the full OpenAlex database rather than within the scrape only: The citation_normalized_percentile.value field is assumed to have been computed by OpenAlex against all papers in each field and publication year in its database, not just the 2,245 papers in this scrape. This assumption holds because the percentile is a pre-computed OpenAlex field rather than one derived from the scrape. If it were computed within the scrape only, the 95th percentile threshold would reflect standing within 2,245 records rather than the full OpenAlex population, making it a far weaker measure of genuine citation impact.

Implementation:

ICLRdata was filtered to papers with non-empty score arrays and a mean score below 5, computed using F.aggregate()/F.size(). OpenAlex was deduplicated using a Window function retaining the highest percentile record per title before inner joining. Papers were then filtered to those in the 95th percentile using the pre-computed citation_normalized_percentile.value, rather than calculating a threshold from the small matched subset.

```
# ============================================================
# QUESTION 6 — LOW SCORE MATCHED PAPERS IN THE 95TH CITATION PERCENTILE
# ============================================================
# Uses ICLRdata filtered to non-empty scores. No decision required.
# mean_score < 5 defines a low score paper.
# citation_normalized_percentile.value >= 0.95 used as the pre-computed
# 95th percentile indicator from OpenAlex.

# Compute mean score per paper and filter to low-score papers
iclr_q6 = ICLRdata \
    .filter(F.size(F.col("scores")) > 0) \
    .withColumn("mean_score",
        F.aggregate(
            F.col("scores"), F.lit(0.0),
            lambda acc, x: acc + x.cast("double")
        ) / F.size(F.col("scores"))
    ) \
    .filter(F.col("mean_score") < 5)

print(f"Papers with mean score < 5: {iclr_q6.count()}")
```

```
# Prepare OpenAlex: extract title, citation count and normalised percentile value.
# Deduplicate by keeping the record with the highest percentile value per title.
# This ensures each title retains a single representative percentile record.
w_oa = Window.partitionBy("title")
openalex_q6 = openalex_raw \
    .select(
        F.col("title"),
        F.col("cited_by_count").cast("double"),
        F.col("citation_normalized_percentile.value").alias("percentile_value")
    ) \
    .withColumn("max_percentile", F.max("percentile_value").over(w_oa)) \
    .filter(F.col("percentile_value") == F.col("max_percentile")) \
    .drop("max_percentile") \
    .dropDuplicates(["title"]) \
    .withColumnRenamed("title", "oa_title")

print(f"OpenAlex records after dedup: {openalex_q6.count()}")

# Join low score ICLR papers to OpenAlex on exact title match
q6_joined = iclr_q6.join(
    openalex_q6,
    iclr_q6.title == openalex_q6.oa_title,
    how="inner"
).select(
    iclr_q6["id"],
    iclr_q6["title"],
    iclr_q6["year"],
    "mean_score", "cited_by_count", "percentile_value"
)

print(f"Low score papers with OpenAlex citation data: {q6_joined.count()}")
```

```python
# Filter to papers in the 95th percentile using citation_normalized_percentile.value
q6_result = q6_joined \
    .filter(F.col("percentile_value") >= 0.95) \
    .select("id", "title", "year", "mean_score", "cited_by_count") \
    .orderBy(F.col("cited_by_count").desc())

print("Q6 Answer — low score papers in the 95th citation percentile:")
q6_result.toDF("ID", "Title", "ICLR Submission Year", "Average Score", "Cited By Count").show(truncate=False)
print(f"Total papers identified: {q6_result.count()}")
```

Results:

```
Papers with mean score < 5: 15045
OpenAlex records after dedup: 1219
Low score papers with OpenAlex citation data: 62
Q6 Answer — low score papers in the 95th citation percentile:
+----------+-----------------------------------------------------------------------------------------------------+--------------------+-----------------+--------------+
|ID        |Title                                                                                                |ICLR Submission Year|Average Score    |Cited By Count|
+----------+-----------------------------------------------------------------------------------------------------+--------------------+-----------------+--------------+
|S19eAF9ee |Structured Sequence Modeling with Graph Convolutional Recurrent Networks                             |2017                |4.0              |763.0         |
|HkmaTz-0W |Visualizing the Loss Landscape of Neural Nets                                                        |2018                |4.666666666666667|514.0         |
|H1bM1fZCW |GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks              |2018                |4.666666666666667|313.0         |
|HJWGdbbCW |Reinforcement and Imitation Learning for Diverse Visuomotor Skills                                   |2018                |4.666666666666667|209.0         |
|ByexElSYDr|Fair Resource Allocation in Federated Learning                                                       |2020                |4.0              |132.0         |
|Hy-lXyDWG |Incremental Learning in Deep Convolutional Neural Networks Using Partial Network Sharing             |2018                |3.3333333333333335|128.0         |
|HJeq43AqF7|Unsupervised Latent Tree Induction with Deep Inside-Outside Recursive Auto-Encoders                 |2019                |4.333333333333333|111.0         |
|S1xFm6VKDH|Meta Module Network for Compositional Visual Reasoning                                               |2020                |3.0              |60.0          |
|H1emfT4twB|Few-shot Text Classification with Distributional Signatures                                          |2020                |3.3333333333333335|54.0          |
|B1esx6EYvr|A critical analysis of self-supervision, or what we can learn from a single image                    |2020                |4.333333333333333|51.0          |
|BkPrDFgR- |Piecewise Linear Neural Networks verification: A comparative study                                   |2018                |4.666666666666667|49.0          |
|HylvlaVtwr|Learning to Sit: Synthesizing Human-Chair Interactions via Hierarchical Control                      |2020                |2.3333333333333335|20.0          |
|Byg5flHFDr|EvoNet: A Neural Network for Predicting the Evolution of Dynamic Graphs                              |2020                |2.3333333333333335|13.0          |
+----------+-----------------------------------------------------------------------------------------------------+--------------------+-----------------+--------------+
```

```
Total papers identified: 13
```

Discussion:

Of the 15,045 papers in the ICLR dataset with a mean score below 5, only 62 had a matching OpenAlex record. Only 13 of these 62 matched papers fell in the 95th citation percentile. Additionally, these 13 papers are only submissions from 2017 to 2020. Question 5 revealed that the overall OpenAlex match rate to the ICLR dataset drops rapidly from 2022 onwards and reaches 0% from 2024-2026. This suggests that the low-scoring and highly cited submissions from 2021-2026 are missing entirely from the OpenAlex scrape. Therefore, the committee should treat 13 as a minimum estimate.

The most cited paper, “Structured Sequence Modeling with Graph Convolutional Recurrent Networks”, has 763 citations despite having a mean score of 4.0. This suggests that reviewer scores, while strongly predictive of acceptance as shown in Question 1’s extension, do not solely predict a paper’s academic value alone.

The extension aims to determine whether, among all 62 matched low-scoring papers, rejected papers accumulate more citations than accepted ones. If so, it would suggest that the review process may be rejecting impactful work more often than previously recognised.

Extension Assumptions:

Mean citation count is an appropriate summary statistic for comparing the two groups: The extension utilises the mean number of citations as a statistic to compare the groups. However, there are only 24 accepted and 38 rejected low scoring papers, with one rejected paper accumulating 763 citations. Therefore, a single paper is pulling the rejected group's average upward. This assumption partially holds. The mean still shows the overall direction (rejected papers accumulated more citations on average), but the ICLR committee should focus on the direction of the gap and not the exact numbers.

Extension Implementation:

ICLRdata was left joined on id to retrieve decisions, preserving all core results. Papers were classified using the same .rlike() pattern as all other questions, with withdrawn papers included in the rejected group. A groupBy() aggregated count, mean, minimum, and maximum citations per outcome.

```
# ============================================================
# QUESTION 6 EXTENSION — CITATION IMPACT: ACCEPTED VS REJECTED
# LOW-SCORING PAPERS IN THE OPENALEX MATCHED SET
# ============================================================
# Among all 62 low-scoring papers with OpenAlex citation data,
# do accepted papers accumulate more citations than rejected ones?

# Join q6_joined (62 papers) to decisions from ICLRdata
q6_decisions = q6_joined.join(
    ICLRdata.select("id", "decision"),
    on="id",
    how="left"
).withColumn("outcome",
    F.when(
        F.col("decision").rlike("(?i)accept|oral|poster|spotlight|workshop|talk|notable"),
        "Accepted"
    ).otherwise("Rejected")
).filter(F.col("decision") != "")

print(f"Matched papers with a mean score < 5 and have a known decision: {q6_decisions.count()}")

q6_decisions.groupBy("outcome").agg(
    F.count("id").alias("papers"),
    F.round(F.avg("cited_by_count"), 1).alias("mean_citations"),
    F.round(F.min("cited_by_count"), 0).alias("min_citations"),
    F.round(F.max("cited_by_count"), 0).alias("max_citations")
).orderBy("outcome") \
    .toDF("Outcome", "Number of Papers", "Average Cited By Count", "Minimum Cited By Count", "Maximum Cited By Count") \
    .show(truncate=False)
```

Extension Results:

```
Matched papers with a mean score < 5 and have a known decision: 62
+--------+----------------+----------------------+----------------------+----------------------+
|Outcome |Number of Papers|Average Cited By Count|Minimum Cited By Count|Maximum Cited By Count|
+--------+----------------+----------------------+----------------------+----------------------+
|Accepted|24              |40.3                  |0.0                   |514.0                 |
|Rejected|38              |55.6                  |0.0                   |763.0                 |
+--------+----------------+----------------------+----------------------+----------------------+
```

Extension Discussion:

The extension achieved its aim. Across the 62 matched low-scoring papers with a known decision, rejected papers averaged 56 citations compared to 40 citations for accepted papers. This shows that among the low-scoring papers, the ones that the committee rejected went on to have greater academic impact than those it accepted. This builds on Question 1's extension. While reviewer scores reliably predict which papers are accepted, they appear to have a weaker influence on which papers the academic community will later judge as valuable.

These findings should be interpreted with caution. The sample of 62 papers is small and entirely from 2017 to 2020 submissions. This means that these results cannot be applied to more recent submissions.

A useful future step would be to build a prediction model that identifies which low-scoring papers are likely to become highly cited. A random forest model using features such as score variance, number of reviewers, submission year, keywords, and number of references could predict citation impact and flag papers for second review before rejection. This model is currently not feasible, as only 62 low-scoring papers have citation data. Pulling citation counts from an academic database such as Semantic Scholar and applying them to the ICLR dataset would make this feasible.